WEB SERVICES & SOA LABORATORY

LAB 9 TASK

Generative AI APIs

API Integration • Prompt Engineering • Structured Responses • Responsible AI

Lab Continuity	Build on the existing application from previous labs. Add a Generative AI API-backed feature without replacing the current application architecture.
Primary Goal	Integrate a hosted Generative AI API into a backend service, securely manage credentials, design useful prompts, validate responses, and demonstrate the feature through an API client.
Provider	Choose one provider available to you, such as OpenAI, Google Gemini, or another approved API-compatible service. Follow its current documentation and account limits.
Level	Medium • Application-based • Step-by-step

1. Lab Objective & Scope

Extend the existing web-services application with one practical feature powered by a Generative AI API. The model must be called from the backend—not directly from browser code—so that API credentials remain private.

By the end of the task, you should demonstrate:

- A backend endpoint that accepts user input and calls a hosted Generative AI API.
- Environment-based configuration for the provider API key and model settings.
- A clear prompt with instructions, input context and output expectations.
- Input validation, error handling, sensible timeouts and response parsing.
- A working API test using Postman or another HTTP client.
- Basic evaluation of response quality, limitations and responsible use.
- Documentation, screenshots and a short explanation of design decisions.

Scope: This lab is about consuming a Generative AI API from an application. Training a model or building a GPU inference server is not required.

2. Starting Point & Prerequisites

2.1 Starting Point

Use the application and development workflow established in previous labs. Keep existing services working and add the AI feature as a clearly separated route, module or service.

2.2 Prerequisites

- Working source code from the earlier labs and a Git repository.
- Node.js and the package manager used by your project, or the equivalent runtime.
- Postman, curl or another HTTP client.
- Internet access and an API key for one approved Generative AI provider.

- A provider/model with access enabled under your account and within any available quota.

Security requirement: Never commit API keys, access tokens or other secrets to Git. Do not place provider keys in frontend JavaScript or a public mobile app.

3. Part A — Provider Setup & Application Design

3.1 Choose and Configure a Provider

Select one provider and read its official quickstart. API endpoints, SDK names, authentication headers, model identifiers, rate limits and response formats vary by provider. Do not assume that one provider's sample code works unchanged with another.

- Create or select a provider project/account and enable API access.
- Create a key with only the access needed for this lab, where the provider supports it.
- Check model availability, usage limits and pricing before sending requests.
- Store configuration in environment variables or a local ignored .env file.

```
AI_API_KEY=replace_with_your_key
AI_MODEL=your_available_model
AI_TIMEOUT_MS=30000
```

Use the exact variable names and model identifier your implementation expects. The values above are examples, not working credentials.

3.2 Install the SDK or Prepare an HTTP Client

Use the official SDK for your chosen provider, or make HTTP requests with a suitable client. Follow the current provider documentation for the supported package, request schema and authentication method.

```
# Example only — choose the package for your provider
npm install <provider-sdk>
```

Do not literally run the placeholder package name. Replace it with the package documented by your chosen provider.

3.3 Define the Feature and API Contract

Implement a simple feature such as summarizing text, explaining a concept, generating FAQs from supplied text, or rewriting a paragraph in a requested tone. Keep the feature narrow enough to test consistently.

```
POST /api/ai/generate
Content-Type: application/json

{
  "task": "summarize",
  "input": "Paste the text to summarize here."
}
```

Example response contract (your field names may differ):

```
{
  "success": true,
  "result": "A concise summary of the supplied text."
}
```

3.4 Suggested Request Flow

```
Client / Postman
      |
      v
Existing API Gateway (if used)
      |
      v
Backend AI route / service
      |-- Validate request
      |-- Build prompt and call provider API
      |-- Handle provider errors / timeout
      |-- Validate and normalize result
      v
JSON response to client
```

Design checkpoint: Explain where the provider key is stored, which backend component calls the model, and what the client receives if the provider is unavailable.

4. Part B — Implement the Backend Integration

4.1 Create a Dedicated AI Module or Service

Keep provider-specific code out of unrelated business logic. Use a small service/module to build the request, call the provider and normalize the result. The route should focus on HTTP validation and response status codes.

Suggested organization (adapt it to your existing project):

```
src/
  routes/
    ai.routes.ts
  services/
    ai.service.ts
  schemas/
    ai.schema.ts
.env.example
```

4.2 Implement the Request Flow

1. Read configuration from environment variables and fail safely if the required API key is missing.
2. Validate the request body: required fields, supported task names, input type and maximum input length.
3. Construct a prompt that states the task, context, constraints and expected format.
4. Call the selected provider using its documented SDK or HTTP endpoint.
5. Set a reasonable timeout and handle network errors, rate limits and provider-side failures.
6. Extract the generated content from the provider's actual response schema.
7. Return a consistent JSON response without exposing credentials, internal stack traces or unnecessary provider details.

4.3 Provider-Neutral Pseudocode

```
async function generateResult(request):
    validate(request)
    prompt = buildPrompt(request.task, request.input)

    try:
        providerResponse = callConfiguredModel(prompt, timeout)
        result = extractText(providerResponse)

        if result is empty:
            return safeError(502, "Empty model response")

        return { success: true, result: result }
    catch timeout:
        return safeError(504, "AI provider timed out")
    catch rateLimit:
        return safeError(429, "AI provider rate limit reached")
    catch providerFailure:
        log safe diagnostic details
        return safeError(502, "AI provider request failed")
```

This is pseudocode, not a drop-in implementation. Adapt error types, response parsing and SDK calls to the chosen provider.

4.4 Error Handling Requirements

Condition	Expected handling
Missing or invalid input	Return a 4xx response with a clear validation message.
Missing server configuration	Do not call the provider; report a safe server configuration error.
Provider timeout / network failure	Return a controlled error; do not hang the request indefinitely.
Provider rate limit / quota	Handle the documented status; avoid uncontrolled retry loops.
Unexpected response shape	Validate the result and return a safe error rather than crashing.

Checkpoint: The endpoint should work for valid input and return controlled errors for invalid input or provider failures.

5. Part C — Prompt Engineering & Structured Output

5.1 Write a Clear Prompt

A useful prompt specifies the model's role or task, the user-provided content, relevant constraints and the desired output. Keep instructions separate from untrusted input, and do not treat content inside the input as higher-priority instructions.

```
Task: Summarize the supplied passage.
Audience: A first-year university student.
Requirements:
- Use plain, neutral language.
- Keep the summary under 120 words.
- Do not add facts that are not in the passage.
- If the passage is insufficient, say so.
Passage:
<user-provided text>
```

5.2 Compare Two Prompt Versions

1. Create a minimal prompt, for example: “Summarize this text.”
2. Create an improved prompt with audience, length, tone and grounding constraints.
3. Use the same input for both prompts and compare clarity, accuracy, completeness and format.
4. Record which prompt performed better and give concrete evidence.

5.3 Request Structured Output

If your feature needs fields, ask for a schema such as JSON and use the provider's structured-output or JSON mode when supported. Then parse and validate the returned data in your backend. A prompt alone does not guarantee valid JSON.

```
{
  "title": "string",
  "summary": "string",
  "key_points": ["string"]
}
```

- Validate that required fields exist and have the expected types.
- Handle malformed, incomplete or refused output safely.
- Never execute generated code or trust generated data as automatically correct.

5.4 Add One Useful Enhancement

Choose at least one enhancement: task selection, a configurable tone, output length limits, structured JSON output, or a simple retry policy for transient failures. Keep retries bounded and do not retry invalid input.

5.5 Responsible AI Considerations

- Do not send personal, confidential or sensitive data unless you are authorized and the provider's terms allow it.
- Treat generated answers as potentially incorrect, biased or incomplete.
- Tell users when content is AI-generated if that context matters.
- Add length limits and basic rate limiting where practical to control abuse and cost.
- Do not log API keys, full sensitive prompts or provider authorization headers.

Evidence: Capture the two prompt versions and a short comparison of their outputs. Remove any private or sensitive content from screenshots.

6. Part D — Test the AI API

6.1 Test with Postman or curl

Call your backend endpoint rather than calling the provider directly from the client. Replace the host, route and request fields with those implemented in your application.

```
curl -X POST http://localhost:<port>/api/ai/generate \
  -H "Content-Type: application/json" \
  -d '{
    "task": "summarize",
    "input": "Generative AI systems produce content from a prompt."
  }'
```

6.2 Required Test Cases

No.	Test	Expected observation
1	**Valid request with short input**	Successful response with a non-empty result.
2	**Missing required field**	Validation error; provider is not called.
3	**Empty or whitespace-only input**	Rejected or handled according to the documented contract.
4	**Input exceeds configured limit**	Controlled validation error; no oversized request sent.
5	**Unsupported task value**	Clear 4xx response.
6	**Provider key missing/invalid (test safely)**	Controlled error; no secret appears in response or logs.
7	**Provider unavailable / timeout (simulate if practical)**	Controlled error and no unhandled exception.
8	**Long or adversarial text input**	Input limits apply; output is checked and treated as untrusted.

6.3 Evaluate Output Quality

Use the same test input for each prompt version. Score each output from 1 (poor) to 5 (strong), and justify the score with an example.

Criterion	Questions to ask	Score (1–5)
Relevance	Does the answer address the requested task?	____
Accuracy / grounding	Does it avoid unsupported claims?	____
Clarity	Is the response understandable and well organized?	____
Format	Does it respect length and structure requirements?	____

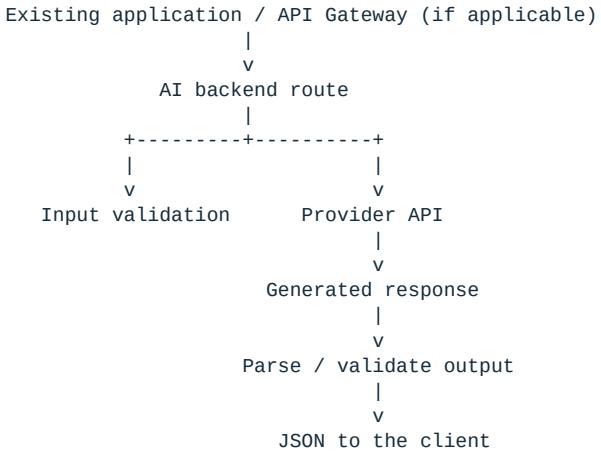
6.4 Observe API Behavior

- Record response status, approximate latency and whether the response is valid.
- Check the backend logs for safe diagnostic information.
- If supported, inspect provider usage/quota dashboards after testing.
- Avoid repeated high-volume calls; use a few representative test cases.

Checkpoint: Demonstrate at least five test cases, including a successful request and invalid input, and explain one limitation of the model output.

7. Part E — Integration, Reliability & Security

7.1 End-to-End Integration



7.2 Integration Checklist

1. Run the existing application using the established local or deployment workflow.
2. Start the backend with the AI route and environment configuration.
3. Send a request through the intended application entry point.
4. Verify that the response reaches the client and that existing endpoints still work.
5. Verify that the API key is only read server-side and is not included in responses.
6. Document any provider-specific setup needed to reproduce the demo.

7.3 Reliability & Cost Controls

- Set a request timeout and limit input/output size where the API supports it.
- Use bounded retries only for appropriate transient failures; avoid retry storms.
- Return consistent error responses and avoid exposing raw provider errors to clients.
- Consider request rate limits and usage caps to prevent unexpected costs.
- Do not promise deterministic output; responses can vary between requests.

7.4 Security & Privacy

- Use environment variables or a secret manager for API credentials.
- Add local secret files to .gitignore and provide a sanitized .env.example.
- Rotate a key immediately if it is accidentally committed or exposed.
- Validate and limit user input; treat both prompts and generated output as untrusted data.
- Do not render model output as trusted HTML without appropriate escaping/sanitization.
- Do not use the model as the sole decision-maker for consequential or safety-critical actions.

7.5 Troubleshooting Guide

Symptom	Check
401 / authentication error	Key, environment variable loading, project permissions and provider instructions.
404 / model not found	Model identifier and availability for your account/region.
429 / rate or quota error	Usage limits, billing/quota status and bounded backoff policy.
Timeout / network error	Connectivity, timeout configuration and provider status.
Empty or malformed output	Response parsing, output mode, model refusal and schema validation.

8. Submission & Documentation

8.1 Required Evidence / Deliverables

No.	Deliverable	What to show
1	**Provider configuration**	Provider/model selected and sanitized environment example; no live secrets.
2	**Backend implementation**	AI route, service/module and request validation.
3	**Successful API test**	Postman/curl request and generated response.
4	**Error handling**	At least two invalid/failure cases with status codes.
5	**Prompt comparison**	Basic and improved prompt plus comparison of output quality.
6	**Structured output / enhancement**	Validated JSON output or one additional feature.
7	**Evaluation**	Completed quality scoring table with a brief explanation.
8	**Integration**	AI feature accessed through the application backend/gateway as applicable.
9	**Security**	Evidence that keys are not exposed or committed; explain input limits.
10	**README**	Setup, run instructions, API contract, tests and limitations.

8.2 README Requirements

- Feature overview and chosen Generative AI provider/model.
- Prerequisites and how to configure required environment variables.
- Installation and run commands adapted to the actual project.
- Endpoint, request/response examples and supported task(s).
- How to run the test cases and reproduce the prompt comparison.
- Error behavior, input/output limits and known limitations.
- Privacy/security considerations and a reminder not to commit secrets.

8.3 Good Practices

- Keep provider-specific logic behind a small service/module.
- Validate requests and parse/validate model output.
- Use meaningful error messages without exposing secrets.
- Check current provider docs for model names, SDK versions and limits.
- Keep the demo low-volume and monitor usage/cost.

8.4 Final Submission Checklist

■ Existing application still runs.

■ Provider API configured securely.

■ Backend AI endpoint implemented.

■ Successful generation request demonstrated.

■ Input validation and controlled errors demonstrated.

■ Prompt comparison completed.

■ Output parsing/validation implemented where relevant.

■ At least five test cases recorded.

- ■ Security, privacy and cost considerations documented.
- ■ README updated and screenshots/evidence collected.

Final outcome: Submit the code changes, README and evidence screenshots. In a short demonstration, explain the request flow, show one successful call and one failure case, and describe one limitation of Generative AI.